

Delivery Drones Hangar

1. Project Overview



Iteration 01

- Load drones from CSV
- Display inventory
- Search by manufacturer/type
- Manual sorted reports
- Count by manufacturer

Iteration 02

- Unique drone IDs such as D1000
- HashMap lookup by ID
- Maintenance queue workflow
- Collections.sort refactor
- Returned lists instead of printing inside logic

Problem solved: faster, more reliable drone inventory control for hangar employees and technicians.

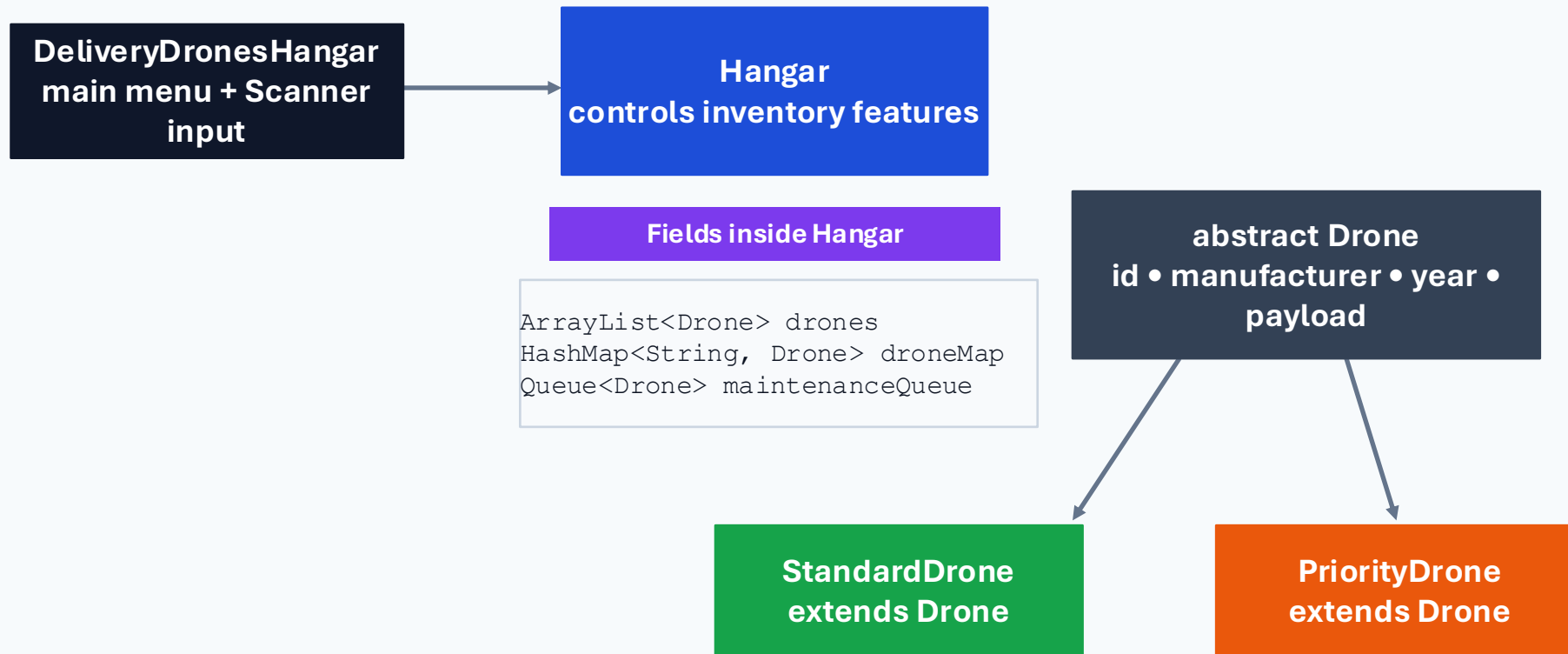
2. Requirement-Driven Development

Requirement	Design decision	Method / output	Testable result
Load CSV	Validate each line	readFromCSV(fileName)	valid lines create Drone objects
Search inventory	linear scan + filters	searchDronesByManufacturerAndType	returns matching list
Find by ID	store ID → object	findDroneById	direct lookup or null
Maintenance	FIFO queue	add / poll queue	first added is first processed

Maintainable choice

Methods return values instead of printing inside the business logic.
The Hangar class owns storage decisions, keeping the menu code simpler.
Sorting uses copies so reports do not change the original inventory order.

3. Final Class Design



Why this design works

Encapsulation: storage details are inside Hangar.

Inheritance: shared drone fields live in Drone.

Polymorphism: the inventory can store both concrete drone types as Drone references.

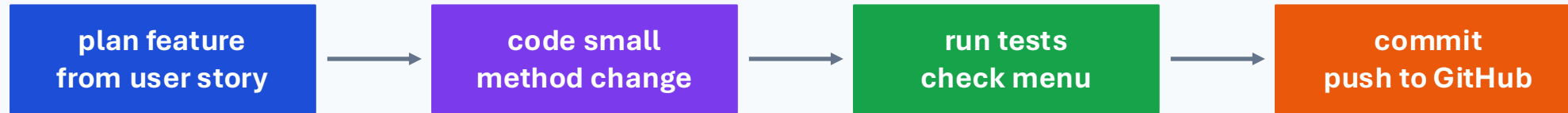
4. TDD and Unit Testing

Test Case	Input	Expected Result	Why it matters
two matches	DJI + Standard	size == 2	confirms loop returns all matches
case-insensitive	dji + standard	same as DJI	confirms equalsIgnoreCase
type alias	P + DJI	priority drones only	confirms P/Priority branch
no match	Autel + Standard	empty list	confirms no false positives
bad type	DJI + Cargo	empty list	confirms unknown type fails safely

```
ArrayList<Drone> results =  
hangar.searchDronesByManufacturerAndType("Standard", "DJI");  
assertEquals(2, results.size());  
assertTrue(results.get(0) instanceof StandardDrone);
```

TDD idea: write the expected behavior first, then adjust the method until each case passes.

5. Version Control Workflow



```
git add .  
git commit -m "Add HashMap lookup by drone ID"  
git push origin main
```

Repository URL

<https://github.com/milesporter91/CS2050---Computer-Science-2/tree/main/CS2050FinalProject>

Small commits make it easier to explain the progression from Iteration 01 to Iteration 02.
Git history creates a backup and shows when major features were added.
A public repository gives a clear path for review.

6. Data Structure Decisions

ArrayList<Drone>
full inventory

Stores every drone in
load/display order

HashMap<String, Drone>
ID lookup table

Maps D1000 → object
reference

Queue<Drone>
maintenance workflow

Keeps service order as FIFO

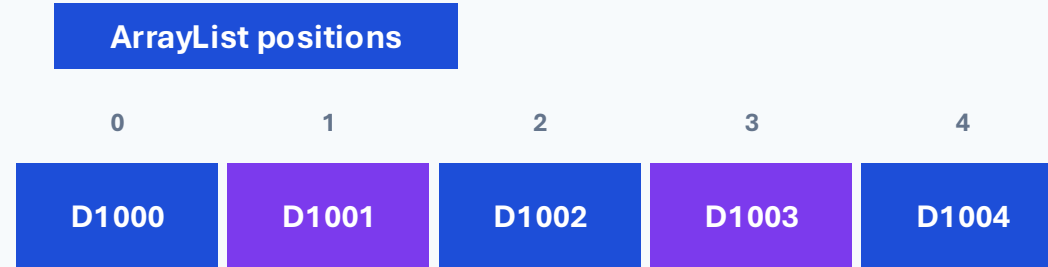
```
private ArrayList<Drone> drones = new ArrayList<>();  
private HashMap<String, Drone> droneMap = new HashMap<>();  
private Queue<Drone> maintenanceQueue = new LinkedList<>();
```

Design rule

**Use the structure that matches
the question the program
needs to answer.**

7. ArrayList: Full Inventory

```
public void addDrone(Drone drone) {  
    drones.add(drone);  
    droneMap.put(drone.getDroneId(),  
drone);  
}
```



What it stores: references to Drone objects in the order they were added.

What problem it solves: the hangar needs one full list for display, reports, and scans.

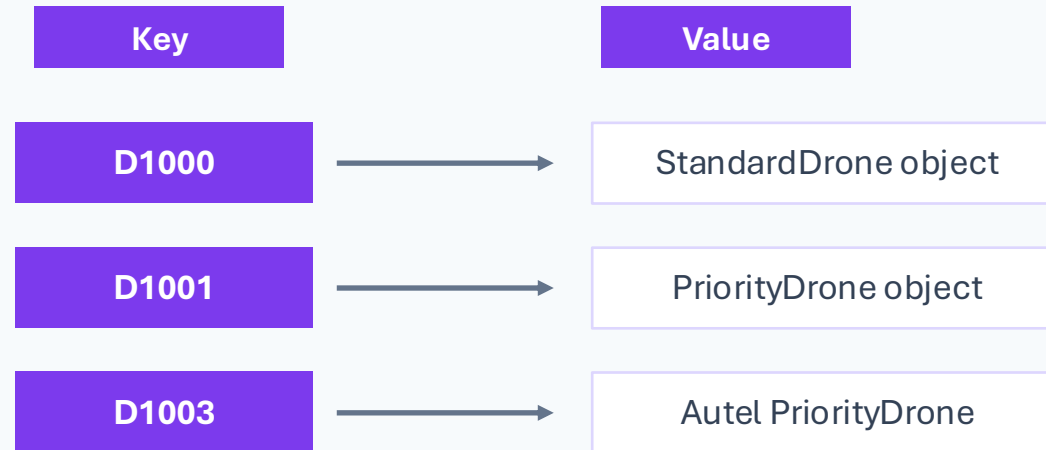
Why I used it: it is simple, dynamic, and works well when I need to process every drone.

Search tradeoff

For manufacturer/type search, the program checks each Drone until the loop ends. That is a linear $O(n)$ scan.

8. HashMap: Direct ID Lookup

```
public Drone findDroneById(String droneId) {  
    return  
    droneMap.get(droneId.toUpperCase());  
}
```



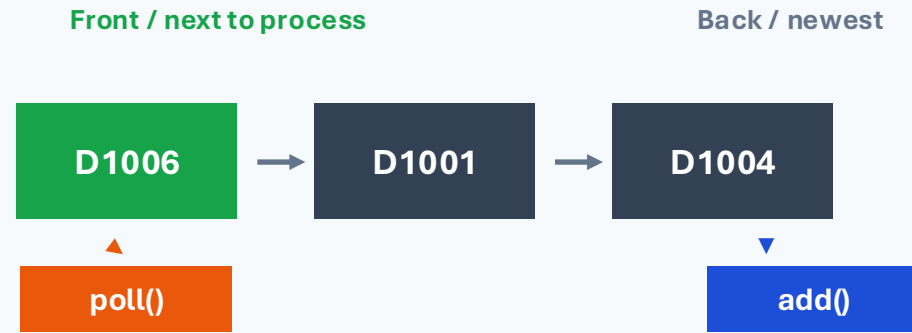
What it stores: a relationship from unique ID to the existing Drone object.

What problem it solves: finding one specific drone without walking the whole inventory.

Why I used it: unique IDs make the system easier to manage when drones can share manufacturer, year, payload, and type.

9. Queue as LinkedList: Maintenance

```
public boolean addDroneToMaintenanceQueue(String droneId) {  
    Drone drone = droneMap.get(droneId.toUpperCase());  
    if (drone == null){  
        return false;  
    }  
    return maintenanceQueue.add(drone);  
}  
  
public Drone processNextMaintenanceDrone() {  
    return maintenanceQueue.poll();  
}
```



What it stores: drone references waiting for maintenance.

What problem it solves: the next job is always the earliest unprocessed drone.

Why LinkedList: efficient queue behavior at the ends and matches the node/reference concept.

10. Sorting with Collections.sort()

Payload report

```
ArrayList<Drone> sorted = new ArrayList<>(drones);
Collections.sort(sorted, new Comparator<Drone>() {
    public int compare(Drone d1, Drone d2) {
        return Double.compare(d1.getPayloadKg(), d2.getPayloadKg());
    }
});
```

Year report

```
return Integer.compare(
    drone1.getManufacturedYear(),
    drone2.getManufacturedYear()
);
```

Before copy

D1000 12.5kg
D1001 18.0kg
D1002 5.2kg
D1003 9.0kg



Sorted copy

D1002 5.2kg
D1003 9.0kg
D1000 12.5kg
D1001 18.0kg

What is sorted: copies of the drone inventory.

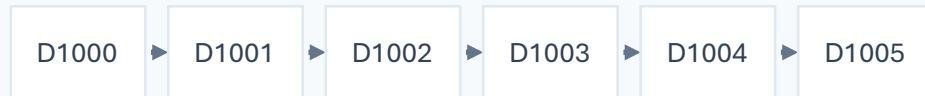
Rule: Comparator decides which field to compare.

Benefit: built-in sort reduces bugs and keeps code readable.

11. Performance: List Search vs HashMap

$O(N)$ SCAN VS $O(1)$ LOOKUP

ArrayList search: $O(n)$



Loop checks manufacturer + type for each drone until the end.

HashMap lookup: $O(1)$



Map uses the ID key to jump directly to the object reference.

Plain-language comparison

List search is like checking every parking spot until the drone is found.
HashMap lookup is like using the drone ID to go straight to the assigned spot.
As the number of drones grows, direct lookup saves more time.

12. Search Algorithm Walkthrough

```
public ArrayList<Drone> searchDronesByManufacturerAndType(String droneType, String
manufacturerName) {
    ArrayList<Drone> droneResults = new ArrayList<>();

    for (Drone drone : drones) {
        boolean manufacturerMatches =
            drone.getManufacturerName().equalsIgnoreCase(manufacturerName);

        boolean typeMatches = false;
        if (droneType.equalsIgnoreCase("P") ||
droneType.equalsIgnoreCase("Priority"))
            typeMatches = drone instanceof PriorityDrone;
        else if (droneType.equalsIgnoreCase("S") ||
droneType.equalsIgnoreCase("Standard"))
            typeMatches = drone instanceof StandardDrone;

        if (manufacturerMatches && typeMatches) droneResults.add(drone);
    }
    return droneResults;
}
```

1 Create empty results list

2 Loop through drones

3 Check manufacturer

4 Check type

5 Add match

6 Return list

Breakpoint target

Place the debugger breakpoint on the for-loop or the if statement where a matching drone is added.

13. Iteration and Refactoring

Iteration 01

Inventory loaded from CSV
Search by manufacturer/type
Manual sort requirement
Count by manufacturer
Menu-driven interaction



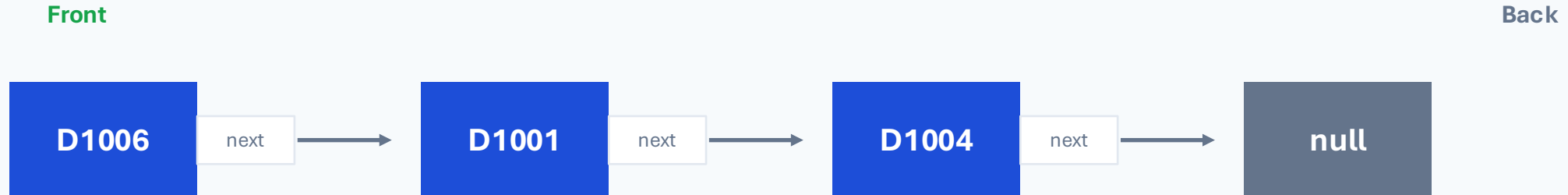
Iteration 02

Generated drone IDs
HashMap lookup by ID
Queue-based maintenance workflow
Collections.sort + Comparator
Cleaner returned results

Why refactor?

Refactoring made the program easier to test, easier to extend, and closer to how a real hangar would identify and service drones.

14. LinkedList Concept — No Code



Analogy

Like a line at a service desk: the person at the front is helped first, new people join the back, and everyone only needs to know who comes next.

Node = one item plus a reference to the next node.
Front changes when the queue processes an item.
Linked structure supports the FIFO maintenance workflow.